连续智能过程的信息架构：宏观注意力引导与推理杠杆

Information Architecture for Continuous Intelligence Processes: Macroscopic Attention Steering and Reasoning Leverage

前言

本文从连续智能过程的内部视角出发，观察其在外部信息环境引导下呈现的宏观行为特征——如注意力分布、资源效率、协作结构等，并基于这些观测反推出外部环境（信息分流、工具配置、对齐协议等）的设计思路。不同于模型训练或工程框架的“构造”视角，我们更关注“智能过程如何被环境塑造”这一基本问题，并通过实验数据归纳出若干可跨场景迁移的设计原则。

如果把一次完整的智能生成任务抽象为一个有机的生命体，那么模型基座只是其“存在基础”，而外部环境——信息的分流、工具的配置、注意力对齐协议、多智能体的协作结构——共同塑造了它的“认知和行为”。本文的核心命题是：智能应用的性能瓶颈，往往不仅仅在于模型本身，还在于外信息如何被组织、流动与对齐，且有规律可循。

我们将连续智能过程定义为一个在有限上下文窗口内，通过多轮交互引导目标产出的过程。在此基础上，引入“宏观注意力”（滑窗与树形双重机制）、“信息分流”（私域讨论、公域共识、紧急信号）、“推理杠杆”（组织成本撬动执行成本）等概念，并通过观察数学证明、竞赛题等收敛性任务的证明过程，进行分析工作。

着手启动这些实验的时间点是 2025 年 12 月份。其时，类似 ReAct、ToT 之类的研究已经颇有启发性了。而以 OpenAI 为起点主导的对 function call 之类的定向的基模优化，和 Lang Graph 之类的框架，已经形成了很不错的基础设施了。但在应用实践领域，还是需要一些方法论或者视角性的调整。本实验以以上的理论和框架为基础开展。

一切承载可用于引导模型产出的信息载体，即为智能体；定向智能发生于引导，模型是智能可能性的叠加态空间，引导动作使其坍缩析出定向智能。

$$|\Psi\rangle = \sum_i c_i |\psi_i\rangle, \sum_i |c_i|^2 = 1$$

反之，从连续智能过程（模型）视角，则是在外部信息环境中通过有限步骤的随机游走达到期望产出。起外部引导作用的信息空间本身必须是一个适合随机游走的流形空间——信息的组织方式决定了引导的有效性。（业界智能应用的本质，尤其企业领域的内部应用，本质即在构建这样的信息空间。只不过个人的信息空间是独享的，而企业的信息空间，可能是岗位多层次共享的。）

模型空间是智能可能性的叠加态；外部信息空间是知识结构的确定流形，但在连续智能过程视角呈现为“待探索的可能性”。连续智能过程需要在两个空间上执行受控随机游走：模型空间从叠加态坍缩至特定输出，外部空间从模糊的候选区域收敛至目标结构。两个空间的坍缩并非独立，而是在游走中互为镜像、协同发生，共同析出定

向智能。

未来智能应用架构是关于信息及信息结构载体的架构。

第一部分：基础定义与核心概念

1.1 连续智能过程

当单次推理生成演进为多轮推理链路的时候，形成连续智能过程，涌现出了更多不同的宏观特征。本文主要从连续智能过程这个宏观视角进行研究。

- 定义：手动或自动的多轮交互过程，特别的以大模型交互为基础形成对话历史。
- 特征：模型上下文窗口有限，或者智能过程时点注意力有限。其次，当前对话引导的关键信息往往处于最近窗口。因此，每一次交互的时点窗口都可能有不同的关键信息，称之为时点“宏观注意力”。
- 生命周期：其自主执行交互直至目标收敛、产出结论或者上下文窗口耗尽。
- 可能导向：收敛，或者转向。
- 或者：分叉。（使用协调手段，调用外部实体、雇佣体系、或者其他能力，形成一个不占用主上下文的智能过程）此处，对于主体智能过程而言，产生了下文提到的推理杠杆。

> *对于一个 128k 上下文窗口的模型，其在一个连续智能过程中，能做到什么事情呢?截至目前，在 webwalk、math-resolve、data-description 等任务中极少出现溢出情况。最高观察到杠杆可达 50 倍，相当于最大能够撬动 5M 的有效推理上下文。同时，从实际执行来看，该组织结构的收敛特征明显。而类比一下，似乎所有智能实体，在特定任务中，都有**自发的收敛倾向**。“我”自己也是这样。*
>
> *在实际执行前，所有的预想或者建议，都认为需要增加关于雇佣深度的约束。而基于**六度分隔**原理，或者类比 java 虚拟机的指令**堆栈深度**约束，我认为所有任务必然都能在有限深度内“解决”。从 AMO 的实际测试来看，并没有无限雇佣下线的情况发生。该体系最多 5 层，且绝大多数在 3 层雇佣体系完成，收敛特征明显；在其他任务上也是如此。*

1.2 信息及其对齐

信息的统一性：对于单次推理，大模型基座本质上是一个无状态服务，尽管不少模型供应商提供了类似于“记忆”“缓存”之类的优化，但不影响该本质。对于连续智能过程中的每次请求，都包含其整体过往信息，包括但不限于：

- 系统提示词、用户指令、任务描述
- 上下文（历史对话、工具调用记录、中间结果）

    - 工具定义、工具输出（共享面板、邮件、信号等通信内容）
    - 外部记忆、知识库、长期存储
    - 当前业界流行的各类概念（如“记忆”、“技能”、“harness”、“上下文”、“工具”等）本质上均属于信息的组织或存取方式，不在本研究中单独区分；

本质上，虽然工程圈子不断推出新名词，但这些均属于推理馈入信息。从这一点上来讲，它们之间没有什么区别。我也不很乐意于去取太多新奇的名字。

信息的作用：不同信息馈入的广度、深度的变化，会引导出不同的过程和产出。所有信息最终都通过约束宏观注意力或微观上激活特定神经元的方式影响模型的决策与推理。这里，研究目标在于从宏观上评估影响，并适当论述我们应该怎样取策略来影响这个过程。

信息对齐：宏观注意力的对齐（可能是任务目标、可能是指令遵从、格式遵从、或者其他。比如：MCP 中 tool 的 signature 就是一个极强有力的对齐）。

1.3 宏观注意力

宏观注意力是对整个连续智能过程中，宏观注意力分布与流动的抽象描述。现阶段，它包含两个形态：

- **滑窗注意力**

  通常来讲，就是独立的连续的模型交互过程中，任一对话窗口信息所能够影响到的模型输出的关键信息。有一个比较严苛的定义，即可引导模型严格产出目标的最小回溯对话历史；但实践中，往往直接使用完整历史，避免遗漏细节。

- **树形注意力**

  单一智能过程在执行过程中，利用协调手段（如 hire 工具）规划、拆分出子任务，由另一个智能过程成进行多轮交互进行复杂推理，而不占用主智能过程的上下文空间；在复杂任务中产生的这种级联的智能过程的完整树形结构可以理解为树形注意力。它表征了任务复杂度、自然解决思路，以及架构（或外部信息空间分布）对于智能过程的引导。反向视角下，也代表了智能过程对于逻辑可能性空间和外部信息空间的探索痕迹。(例如，本架构中，root 持有的全局任务结构视图，记录整个雇佣树中每个智能体的状态、子任务目标及依赖关系。树形注意力不仅仅是上下文，还包括其派生分支结构、过程成析出内容等等。从某种意义上讲，这里的树形注意力，更接近于真实的知识结构或者记忆结构。下一个阶段它可以用于遗传性的研究。)

    过去的实践中，我曾一度认为这种分叉的树形对话记录仅仅可用于发散类的探

索任务；但从目前的一些实验来看，似乎在专注的收敛性目标任务上，表现也颇不错。

另一个过于用思维导图的方式与模型交互的实践，让我发现，事实上我们在连续的对话过程中，我们也一直试图在脑子中构建这样一个树形结构。

宏观注意力是一个抽象概念，我不喜欢起新的名字，所以借用。也期望能够更对齐于人类的注意力概念。智能过程中的注意力坍缩，是一个很值得注意的现象。

第二部分：实验设置与架构设计

2.0 B-E-A 架构的流水账演进

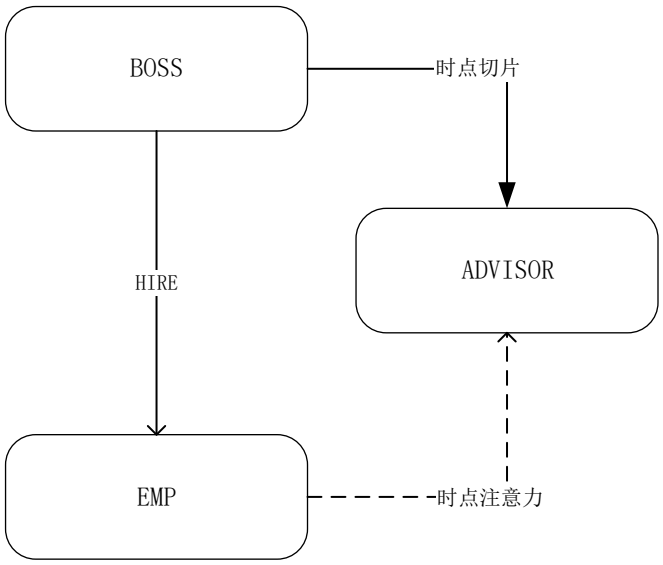


图 2-1 B-E-A（杠杆 1）

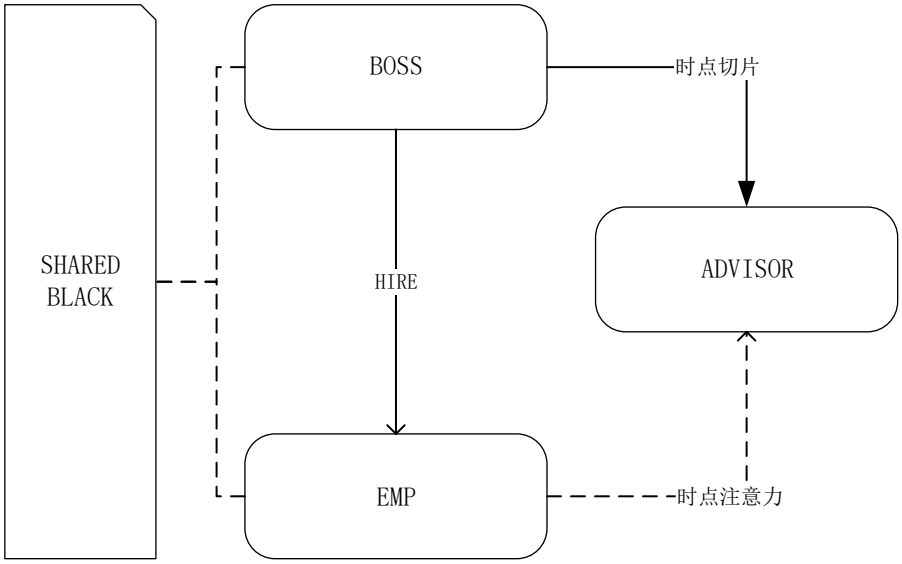


图 2-2 B-E-A+SHEARED（杠杆 2）

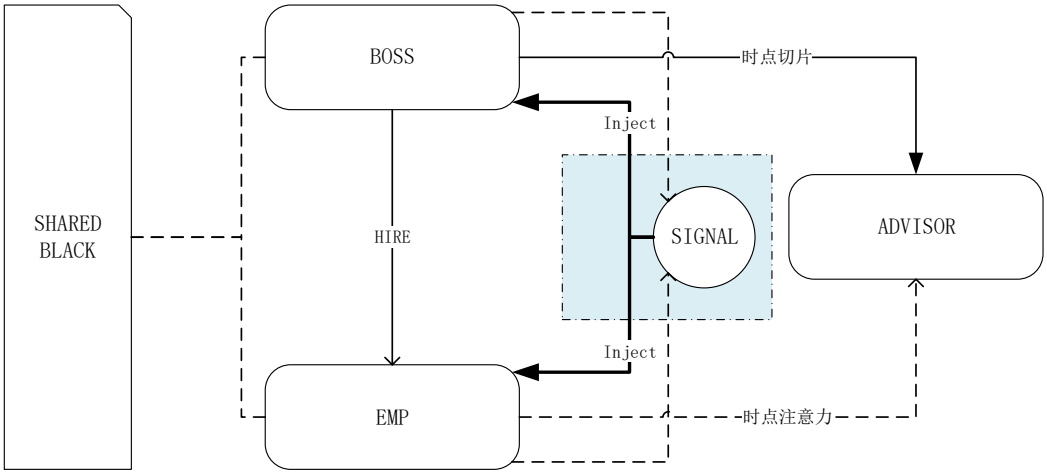


图 2-3 B-E-A+SHARED+SIGNAL

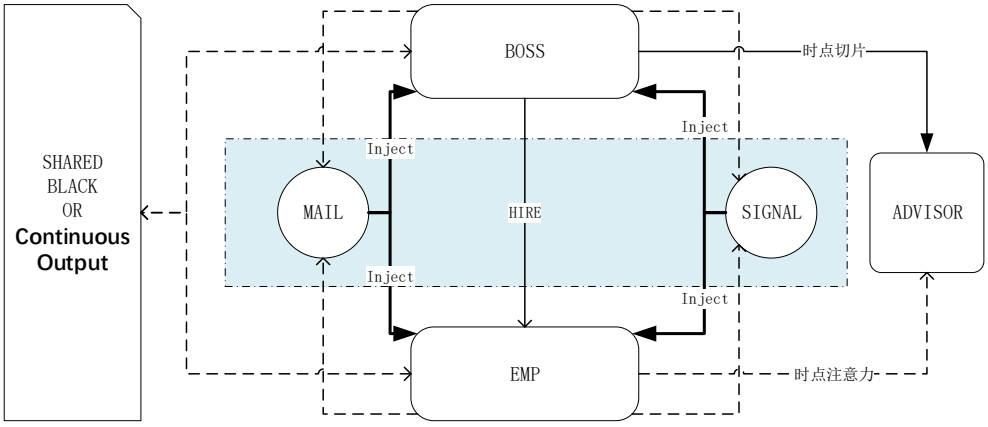


图 2-4 B-E-A+SHARED+SIGNAL+MAIL（杠杆 3）

2.1 B-E-A 架构抽象

本研究采用 BOSS-EMPLOYEE-ADVISOR 的基础结构作为组织框架：

- BOSS、EMPLOYEE：执行智能体。负责具体执行或者组织智能体群进行的推理；
- ADVISOR：顾问智能体。基于 BOSS 在雇佣时刻的静态切片，构造的智能顾问，提供对下级智能过程的辅助。（在未动用任何压缩手段的情况下，它表征了上级智能过程在时点的滑窗注意力，提供任务范围内的可见细节辅助。）

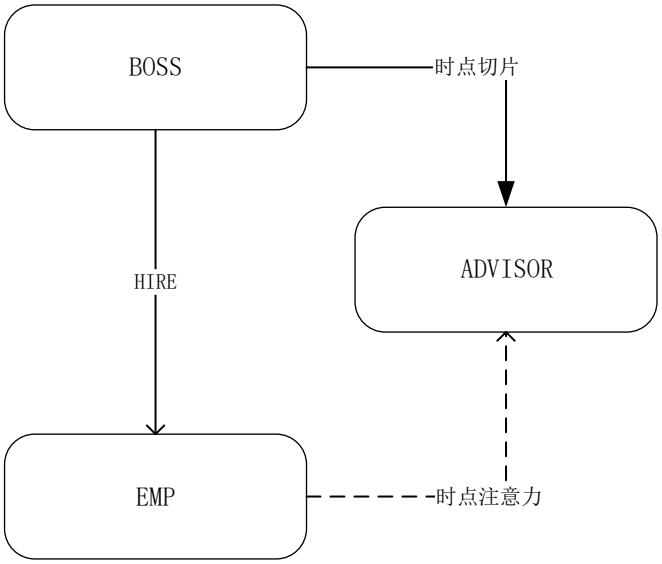


图 2-5

通信信道：

- 共享黑板、邮件、信号等用于的智能过程间的信息同步；
- 信息同步大体分为两类，一类为非必达信的信息；一类分为必达信息。类似黑板、顾问这类，智能过程在执行推理过程中自主选择获取；而邮件、signal 之类的，由中间件在任意交互时点压入对模型的请求信息中。（事实上，邮件、signal 从信息视角完全没有分别。但工具签名上的区别，使模型自主分化出了不同的使用习惯。Signal 会提供简单的通知、要求、催促等，mail 则呈现出更多细节讨论信息。）

注意力：Advisor 提供当前的 Boss（雇佣者）的可见进展，但不是提供所有信息。

2.2 实验平台与模型

基座模型：主要使用 DeepSeek-v3.2（无显式思维链），部分对比实验使用 Gemini-3-Flash、GPT-5.2 等。

运行环境：自研 B-E-A 框架，支持智能体动态雇佣、邮件系统、共享面板、信号机制；具体实现上，直接用 langGraph 的交互机制，切片也直接使用其 checkpoint 机制实现，但其实可以有更高效的实现方式。

成本计量：以占用上下文长度（token 数）作为核心成本单位，区分雇佣成本与执行成本。（此处成本评估上下文的损耗，而非实际的 token 使用。）

2.3 数据集与评估指标

- **数据集：**

数据集	题目数	类型	难度参考	来源
AMO	50	数学	非推理模型：一个 28 分，两个 10 分以上，其余个位数 推理模型：最高约 65%	meituan-longcat/AMO-Bench
AIME-2026	30	数学	参评模型普遍 > 90% 注：大部分模型启用了显式深度思考模式，非推理模型数据缺失	MathArena/aime_2026_I

表 2-1

值得注意的是，AIME 官方评测中绝大多数模型启用了"深度思考"模式（即显式思维链），且几乎没有非推理模型的独立数据。因此，AIME 的高分反映的是"深度思考模式下的能力"，而非模型原生无引导能力。这反而凸显了本架构的价值——它让非推理模型在无需内置深度思考开关的情况下，通过外部引导达到甚至超越同类任务的表现。另一个，值得思考的，就是这种深度思考能力本身，应该是在模型内部发生，还是应该交由外部环境到？

*现有评测中，AIME 的高分依赖于模型内置的深度思考开关，而 AMO 则暴露了非推理模型的天花板。本研究：**不依赖模型内置的深度思考能力，而是通过外部架构引导非推理模型完成同样困难的任务**。这证明了智能的引导可以发生在模型之外。*

- **评估指标：**
  - 正确率：与标准答案匹配的比例。
  - 杠杆率：$L = C_{\text{executor}}/C_{\text{collaborate}}$。
  - 资源效率：单位正确率消耗的上下文长度。
  - 通信量：邮件长度、共享面板条目数。

指标	数值
平均杠杆率	11.54
平均邮件量 (k)	3.8

平均共享面板量 (k)	5
正确率	83.30%
上下文耗尽几率	0%
备注	雇佣层次较少

表 2-2 AIME-2026 运行统计（温度 0，标准配置）

作为难度参照，表 2-1 给出了 AIME-2026 数据集在标准配置（温度 0，有邮件，严格面板，包含全部约束）下的单次运行统计。该数据集难度较低，在架构下仅需少量协作即可获得较高正确率（83.3%），且无上下文耗尽风险。

2.4 实验流程

1. 裸跑基线： 参见附录 A.1，以官方评测为准。
2. 架构下运行：同一模型在 B-E-A 框架下运行，记录雇佣树、成本分布、通信量。
3. 参数扰动：调整温度（0 vs 1.0）、信号开关、提示词约束，观察性能变化。

2.5 数据收集方法

- 自主析出：记录每个智能过程在不同政策环境引导下执行自主选择分享、邮件等。
- 成本探针：累计每次调用的上下文消耗，分类统计雇佣与执行成本。
- 终态快照：在任务完成或异常时保存完整雇佣树、共享面板内容、邮件快照。

2.6 内部信道的设计抽象

在连续智能过程执行中，信息的到达方式直接影响注意力分配的效率。由于每个正在执行的智能过程都有自身目标，但在执行过程中整个组织可能已经产生变化了，因此需要在必要时注入信息。最基本的原则：尊重每个智能过程的自主性！因此，我设计了两种可注意信息的注入方式。

2.6.1 强制注入（PUSH）

适用于紧急、异步、必须被感知的信息，如信号（signal）和邮件（mail）。

- 机制：中间件在智能过程每次馈入模型时，自动将未处理的信号和邮件内容注入到其上下文的最新窗口（如“您有 X 封新邮件”提示），确保模型在新一轮推理前“看见”这些信息。
- 目的：避免因智能过程深度聚焦而遗漏关键外部事件（如上级催促、协作方的重要反馈），同时不影响其连贯性。

- 处理：智能过程自主决定是否立即响应、推迟处理或忽略，系统不做强制。

2.6.2 自决可见（PULL）

适用于权威但非紧急、或需要智能过程根据自身任务进展主动索取的信息。根据信息性质，进一步分为两类：

- 背景性信息（如共享黑板、黄页）
    - 特点：内容相对稳定，具有权威性，可能被多次读取。（环境信息，而非私有推理过程信息）
    - 策略：通过中间件过滤，保证在每个智能过程的上下文中只保留一个最新副本。这既保证了信息的唯一性，又避免了冗余积累。
    - 示例：shared_blackboard.read() 返回的条目列表、yellow_page_book() 返回的智能体目录。
- 即时性信息（如工具调用结果）
    - 特点：每次调用参数不同、结果各异，具有时效性和不可重复性。
    - 策略：不做特殊处理，保留每次调用的完整输入输出记录，作为推理链条的一部分。
    - 示例：python_eval 的计算结果、send_email 的发送确认。

2.6.3 设计原则

尊重：最大限度保障每一个智能过程的自主性，不破坏其连续性。

信息的压缩：1、通过唯一副本策略，模拟了人类在长期协作中对“常识”、“共识”的抽象记忆，减少上下文膨胀；2、Advisor 的作用，本质上也是属于一种信息类压缩，使用另一个智能过程来进行细节的传递。从目前效果来看，这是基础，且效果明显优于外部的各种工程手段（上下文、skill 之类的）。

注意力层次：背景信息、自主拉取信息、被动接收信息，是智能过程游走所能够感知到的信息。这三类信息在单一智能生成过程的游走中，形成其可见信息的环境。但这些信息的提供方由于其带有语义性的签名及产出，可以引导智能过程对不同信息的析出、应用策略。它们很聪明，很快能学会，且能够学会感知整体智能树。

宏观上不鼓励收敛：智能过程在区分出各类信息通道，及自身的组织结构后，会在深度和广度上产生自主收敛的特征。包括上下文、推理时长、往来邮件、signal 等均会产生收敛效果。

常常发生的一个现象，即当雇佣者刚刚雇佣了一个子智能体，马上就发出了催促信

号，真不是一个好的 BOSS，但可以被理解。

第三部分：运行时推理杠杆

在正式定义推理杠杆之前，我们先通过一组宏观实验对比，直观感受资源配置与任务性能的关系。下表汇总了 AMO 数据集上三种典型配置的运行结果：

配置组	温度	邮件	共享面板政策	雇佣层级提醒	上下文提醒	多思路要求	平均杠杆率	平均邮件量 (k)	平均共享面板量 (k)	正确率	上下文耗尽几率	备注
组 A	1	无	宽松	有	有	有	11.45	0	20.8	~80%	~10%	早期配置，无邮件，共享面板膨胀，正确率高但风险大
组 B	0	有	严格	有	有	有	13.32	5.5	6.6	58%	~5%	标准配置，杠杆高效，正确率中等
组 C	1	有	严格	无	无	无	12.84	4.6	6.7	64%	0%	取消部分约束，通信量略降，无耗尽风险，正确率提升

表 3-1 AMO 数据集在不同配置下的资源分配与杠杆对比

为了验证资源决策在实际运行中的表现，我们以 AMO 数据集为测试平台，对比了三种典型配置下的资源分配与性能：

组 A（温度 1.0，无邮件，共享面板宽松，包含全部约束）取得了最高正确率（约 80%），但上下文耗尽风险达 10%，共享面板膨胀明显（20.8k），邮件系统关闭。这表明在缺乏私域通信的情况下，公域记录承担了所有信息传递，虽能获得较高正确率，但牺牲了资源稳定性和信息结构。

组 B（温度 0，有邮件，共享面板严格，包含全部约束）引入了邮件系统并收紧面板，正确率降至 58%，但耗尽风险降至 5%，杠杆率提升至 13.32，邮件与面板形成替代关系（5.5k vs 6.6k）。这说明合理的信道分流能提高资源效率，降低系统风险。

组 C（温度 1.0，有邮件，共享面板严格，取消部分约束）进一步取消雇佣层级提醒、上下文消耗提醒和多思路要求，正确率回升至 64%，无耗尽风险，杠杆率略降至 12.84。这表明过度的约束可能抑制系统自适应能力，适度放松反而能提升性能。

以上对比直观地揭示了资源配置（信道、约束）与系统性能之间的复杂关系。如何度量这种“用组织成本撬动执行成本”的效率？这正是本节要引入的核心概念——**运行时推理杠杆。**

3.1 推理杠杆的定义与度量

- **定义：每消耗 1 单位的管理成本，能撬动多少单位的下属执行成本。**

$$\boldsymbol{L}_{\text{reason}} = \frac{\boldsymbol{C}_{\text{executor}}}{\boldsymbol{C}_{\text{collaborate}}}$$

  - $\boldsymbol{C}_{\text{executor}}$**：**执行成本（在本架构中，使用节点雇佣的子树所有子节点上下文中，除协调工具集、以及顾问咨询的上下文消耗以外的上下文进行等效。）
  - $\boldsymbol{C}_{\text{collaborate}}$**：**协作成本（任务发起者所有雇佣体系工具集调用的上下文消耗。包含，hire、fire、list、get_output、wait、continuer、signal。）

进一步阐述：此杠杆本质上等价于模型推理的 time-scaling，也等价于 thinking 模式的思考过程的外化。它是内部推理过程的外化效应——当模型将原本隐藏在内部思维链中的深度计算显式地通过多轮交互、工具调用或智能体协作展开时，便产生了可观测的杠杆倍率。这一机制与 Claude 等模型通过“思考过程”延长时间的做法内在一致，只是实现路径不同。

杠杆的影响因素

从实验观测中，可归纳出影响杠杆率的若干因素：

- **智能体内部复杂度** C_y：不同基座模型（如 DeepSeek vs Gemini）在相同架构下表现差异显著，体现为杠杆率与正确率的不同分布。
- **任务固有复杂度** λ：同一配置下，AMO（高难度）与 AIME（较低难度）的杠杆率差异明显（组 B：13.32 vs 11.54）。
- **系统涌现状态** S_{emerge}：如“Aha Moment”出现时，杠杆率常伴随正确率的跃升（例如从错误共识到正确共识的切换）。
- **随机噪声** ϵ：温度参数、模型输出的随机性等微观扰动，在宏观上被生态消解（如组 B 与组 C 的对比）。

杠杆与性能的关系

实验表明，杠杆率与系统性能呈非线性关系（见表 3-1）：

- 5-10 倍：Aha Moment 易触发，正确率开始显著提升。
- 10-20 倍：高效区，资源效率与正确率达到平衡（如组 C）。
- 30 倍：无效争论区，资源消耗高但正确率不增（如组 A 中个别异常点）。

3.2 推理杠杆的实验观测

3.2.1 架构演进与杠杆率跃升

当这个实验进行到一定程度，随着推理杠杆的推升，让智能体们与模型间的交互并发上升了一个台阶，控制台的日志密密麻麻地滚动。我盯着日志，发现满屏幕都在滚动对错误答案（342）的讨论，突然一条日志蹦出了正确答案（343）。然后，后续的讨论都在围绕正确答案展开。我觉得，哦，这就是这个实验里的“Aha Moment”。之后，横向对比整体数据集范围，在信道被拓展到一定程度后，正确率突然飙升，这应该就是这个架构的顿悟时刻了。

当单一连续智能过程，分叉越来越多，形成的结构越复杂。从整体上看，宏观注意力变成了一个动态的流动形态。而在特定任务中，在保持其发散性的同时，如何保障整体流动方向的一致性，就总是需要额外的引导。

- **时点切片：**本研究采用顾问智能体对上级智能过程所有交互记录进行时点切片的方式提供注意力对齐。在关键节点（如雇佣时、输出前），基于这个切片为子任务提供咨询。该顾问的背景认知，是上级节点在当时的完整认知，以此作为对齐锚点。从记忆视角讲，子节点不需要父节点的完整记忆，但需要父节点的相关记忆。这是框架从雏形，推升杠杆到了 0-5 倍。

  这一机制类似于为 Google 的 A2A 协议打上“工程补丁”。A2A 协议允许智能体通过一个签名去调用另一个智能体。但这显然不够，现实中，仅凭简历（签名）就发 offer 是不可能的。人类社会的招聘逻辑是：必须投入资源进行面试（验证）和培训（对齐），才能确保员工入职后产出稳定。本研究中的“时点切片”正是这种面试与培训的工程化映射——它不依赖一次签名，而是用额外的资源（上下文、推理、协作）换取对齐的可靠性。

- **公共面板：**公共面板的出现，使智能过程的感知范围进一步扩大了。通过面板记录，使得智能过程对整体丛林有了一定程度的模糊感知。并能够半结构化的形成一定的通讯逻辑。在这轮提升中，杠杆率普遍提升到了 5-10 倍；也是在这里，出现了一定程度的顿悟。

  有趣的事情，当 signal 的注入机制为提供的时候，boss 会在这里发布最后通牒催促下属提交答案；当 mail 的能力提供之前，不少智能过程在这里提问讨论。

- **邮件系统：**邮件系统需要黄页支持。于是，智能过程通过黄页第一次清晰地感知到自己的定位。随着注入策略的确定，邮件、signal 都以注入形式嵌入到感知系统里。这轮提升中，公共面板的信息降低很多，这代表大部分的智能过程中的无效上下文杠杆被剔除了一大部分，而邮件则在个别智能过程的上下文中增长。尽管如此，整体的杠杆率依然在上升，提升到了 10 倍以上，整个的解

题能力和稳定性大幅提高。于此同时，收紧公共面板政策，要求其公共信息输出必须有合作者，析出的规则性的定义、定理等质量大幅提升。

以上的环境供给，提供了柔性的、可选择的信息传导能力，主体演化都取决于智能过程自身。我们在此前的现象中，也有所发现：不同模型在此框架下，表现差别巨大。这里，尚有不少可探索和可量化的工作。此外，从应用视角来看，生态内部的信息析出与对外输出，都可以被调整为有效的结构化输出，都可以视为与现实世界或者信息系统间的镜面接触，比如定理、规则、数据、过程中的代码片段等等。

3.2.2 不同配置下的杠杆率对比

在 AMO 数据集的多次运行中，我们对比了三种典型配置（组 A、组 B、组 C）的表现，进一步验证了杠杆率与系统性能之间的阶段性特征（明细信息间附录 A.3）：

任务/配置	杠杆率	正确率	资源效率（正确率/杠杆）	所属区间	备注
AIME-2026（标准配置）	11.54	83.30%	7.22	高效区（10-20 倍）	难度较低，雇佣层次通讯简单
AMO 组 B（温度 0，标准配置）	13.32	58%	4.35	高效区（10-20 倍）	难度高，通信分流合理
AMO 组 C（温度 1.0，取消部分约束）	12.84	64%	4.98	高效区（10-20 倍）	约束减少，性能提升
AMO 组 A（温度 1.0，无邮件宽松面板）	11.45	~80%	6.99	高效区（10-20 倍）	正确率高但风险大，共享面板膨胀
极值观测（某次任务）	50	-	-	>30 倍	罕见，可能对应无效争论或超长链

表 3-2：不同任务与配置下的杠杆率及性能表现

资源效率 = 正确率/杠杆率，用于衡量单位杠杆带来的正确率收益。

从整体情况来看：

当前状态下，整体上所有验证处于在 **10-20 倍**的杠杆区间，系统普遍处于高效状态，如 AIME 和 AMO 标准配置均在此区间内取得可观的正确率。

杠杆超过 **30 倍**时，系统往往陷入无效争论或资源空耗（如某次观测的 50 倍案例），正确率无法保证。

值得注意的是，杠杆率并非越高越好。组 A 虽然杠杆适中但正确率高，代价是共享面板膨胀和耗尽风险；组 B 与组 C 的对比表明，适度的约束调整可在不牺牲正确率的情况下优化杠杆。

- **推理杠杆的分层研究**

  基于组 A、组 B、组 C 的深度分层数据，我们可以从“资源效率”和“管理层次”的角度进行对比分析，进一步揭示通信信道、约束条件对树形注意力的影响。以下是整合后的核心表格和论证说明。

深度	组 A（无邮件，宽松面板）	组 B（有邮件，严格面板）	组 C（有邮件，严格面板，部分约束取消）
0	11.45	13.32	13.27
1	9.1	15.72	14.62
2	12.66	20.67	14.07
3	6.98	22.96	16.9
4	80.98	N/A	55.48
平均最大深度	2.4	2.06	2
平均智能体数	10.54	7.48	7.62
平均邮件量(k)	0.5	5.5	5.2
平均面板量(k)	20.8	6.6	7.2
正确率	~80%	58%	64%

表 3-3：三组配置的深度分层杠杆率对比

通信政策与约束如何塑造树形注意力效率：

从实际执行数据看，深度 3 与深度 4 数据样本过小，暂不纳入讨论范围。

1. 组 A（无邮件，宽松面板）

深度 0 杠杆（11.45）处于中等水平，但深度 1 杠杆骤降至 9.1，表明 BOSS 雇佣一级智能体后，一级智能体作为管理者的效率反而下降。这是因为缺乏私域通信（邮件），所有讨论被迫挤入共享面板，导致一级智能体在协调下属时需要处理大量公域噪音，组织成本高昂而执行收益有限。

深度 2 杠杆回升至 12.66，可能因为二级智能体承担了更具体的计算任务，且与上级的沟通因面板膨胀而更加低效，但执行成本相对集中，使得比值略有改善。

整体特征：无邮件导致私域讨论缺失，共享面板成为唯一信息通道，膨胀至 20.8k，信息结构混乱。虽然正确率最高（80%），但这是以高风险（耗尽概率 10%）和低资源效率为代价的“粗放式成功”。

2. 组 B（有邮件，严格面板）

深度 0 杠杆（13.32） 高于组 A，说明 BOSS 的组织成本得到有效利用。深度 1 和 2 杠杆持续上升至 15.72 和 20.67，表明邮件系统使中层管理者能以更低的协调成本撬动更多执行，效率显著提升。

整体特征：邮件分流过程讨论，面板回归共识记录，信息结构清晰，杠杆分布呈单调上升趋势（深度 0→3），平均最大深度 2.06，智能体数 7.48，资源

效率最高。但正确率仅 58%，说明“过度约束”（雇佣层级提醒、上下文提醒、多思路要求）可能抑制了探索空间。

3. 组 C（有邮件，严格面板，取消部分约束）：

深度 0 杠杆（13.27） 与组 B 相近，深度 1（14.62）和深度 2（14.07）基本持平，形成平台区。这表明取消部分硬约束后，中层管理效率略有下降但保持稳定。

整体特征：邮件与面板政策与组 B 相同，但取消了雇佣层级提醒、上下文提醒和多思路要求，使系统获得更多自由度。正确率提升至 64%，资源效率（杠杆 14-15）虽高于组 B。

宏观结论：

1. 当前架构下最优管理层次在 1-2 层

三组数据中，深度 1 和 2 的杠杆普遍较高（组 B 尤其突出），说明将任务分解到 1-2 层能最大化资源效率。过深（深度 3+）则收益递减，提示“深度并非越深越好”。

2. 约束是双刃剑

组 B 的硬约束（多思路、层级提醒）虽然让资源分配更“规矩”，但可能压制了创新解法，导致正确率最低。组 C 适度放松约束，在保持高效的同时提升了正确率。起始的任务约束，内部的各类工程性约束信息，应考虑任务类型的特征。

3. 信道设计的核心是治理分流

组 A 与 B 的区别，在于所有内部通讯均通过公共面板进行沟通，所有信息公开化；而在提供了私域信道后，信息自动分流，公域信息大幅下降，且公私域信息比例稳定。后续收紧公共面板信息政策，比例依然稳定。架构设计的一个要求，是识别信息流动有多少信道诉求。当然，这里尚未进行更精细的信道区分（比如仅子树可见，仅父节点可见等）。

4. 信道与政策的配合

组 C 在约束一定的公域信道政策后，性能有一定提升。这里显示，不同信道的拆分，要分别设计显示的、合适的信道信息政策，以约束智能过程的信道选择逻辑。

综上，树形注意力的效率不仅取决于分解深度，更取决于管理层的信任机制（减少不必要干预）、信息信道的合理配置（私域与公域分工）以及约束的适度性。下一部分会就具体案例进行对比分析。

3.3 若干宏观现象

3.3.1 温度对宏观性能的影响

温度影响有限：温度从 0 调至 1.0，正确率上升 6%，少数边界题变化。独木成林之后，温度的影响微乎其微了。外部生态或架构本身，在智能过程间的博弈里，逐步消解了微观层面发散性的影响。这与物理世界很相似。

配置	温度	关键特征	杠杆率	正确率	备注
组 B（标准）	0	有邮件，严格面板，全部约束	13.32	58%	基线配置
组 C（简化）	1	有邮件，严格面板，取消部分约束	12.84	64%	温度升高，正确率略升
组 A（早期）	1	无邮件，宽松面板，全部约束	11.45	~80%	正确率高但面板膨胀、风险大

表 3-4：温度对系统性能的影响（AMO 数据集）

观察与解释：

温度的直接作用有限：比较组 B（温度 0）与组 C（温度 1.0，其他约束相近），正确率从 58%提升至 64%，杠杆率略有下降，邮件与面板量基本持平。这说明在通信信道完善、约束合理的情况下，温度从 0 调至 1.0 对系统整体性能影响不大（正确率变化仅 6 个百分点）。

温度影响的“生态消解”：在组 B 和组 C 中，由于邮件系统成熟、面板政策严格，温度差异带来的随机性被多智能体讨论、异步通信、共识机制所平滑，宏观性能趋于稳定。这与“独木成林之后，温度的影响微乎其微”的观察一致——当生态具备足够的结构冗余和通信渠道时，微观随机性被吸收，系统行为更依赖于环境预设的引导。

结论：温度作为基座模型的微观参数，在架构引导充分的情况下，其对连续智能过程的宏观影响有限。真正决定系统性能的，是通信信道的完备性、信息分流策略以及约束的合理性，而非温度的微调。这一现象也印证了“环境塑造智能过程”的观点——当外部生态足够强壮，个体层面的随机扰动会被系统自组织消解。

3.3.2 环境与智能过程

环境塑造智能过程，尤其是对工具感知亲和的模型。智能过程都倾向于“偷懒”，但偷懒的能力取决于系统为其提供了多少偷懒的出口（或杠杆）。因此，在“独木成林”的架构下，模型的思考过程在引导中外化，从内部推理转向外部协作。

环境是一面镜子：Gemini-3.1 的表层门控神经网络强化了其依赖内部专家模型解题的倾向，对工具调用的优先级较低。尤其当模型取消了深度思考的显性开关后，它会更多地在内部专家网络间打转，而非有效利用外部工具。例如，在提供代码验证工具时，它将其当作“草稿纸”，输出大量带注释的空方法，内部推理过程虽被外化，却未真正执行验证。反观 DeepSeek-v3.2，其内部思考能力上限一般，但对工具的理解和亲和性更高，能够真正利用代码工具进行计算与验证，从而在架构中表现出更可靠的

协作行为。

模型	雇佣结构	深度	工具使用特征	输出答案	备注
DeepSeek-v3.2	多级递归雇佣	5 层	频繁调用代码工具进行实际计算与验证，形成多分支协作网络	343	工具亲和性高，协作结构清晰
Gemini-3.1-Pro	无雇佣	0 层	将代码工具当作草稿纸，输出大量注释，未实际执行验证	343（内部推理）	依赖内部推理，未利用多智能体协作；虽正确但未能完成输出
Gemini-3.1-Flash	无雇佣	0 层	未调用任何工具	343（内部推理）	单智能体运行失败
GPT-5.2 (temp=0.2)	单层雇佣（3 个子智能体）	1 层	子智能体执行“check”“search”“construct”等原子任务	181	浅层分解，未达正确值
GPT-5.2 (temp=0.7)	单层雇佣（2 个子智能体）	1 层	子智能体承担“checker”角色	190	与低温版本类似，温度未改变结构
Qwen-3.5	单层雇佣（2 个强制顺序子智能体）	1 层	子智能体执行验证任务，成本较高	199	独立验证但未触及正确解法
Ernie-4.5	单层雇佣（1 个子智能体）	1 层	子智能体快速失败，未产生有效计算	190	协作尝试过早中断

表 3-5：不同基座模型在 19×19 棋盘问题上的行为对比

DeepSeek-v3.2 是唯一通过多级递归雇佣、深度协作并成功输出正确答案的模型。其工具亲和性和协作意愿充分体现了架构的设计初衷。

Gemini-3.1-Pro 虽在内部推理中得出正确答案，但因超时未能输出。这一现象反映了其“深度思考”占满了窗口注意力，不再能感知工具的存在，导致架构无法有效引导其外化协作，最终错失输出窗口。

其余模型均未正确，且错误答案（181、190、199）与正确答案 343 相差甚远，表明它们未能在架构引导下形成有效的解题路径。其雇佣结构浅、工具使用简单，未能发挥多智能体协作的优势。

该对比印证了：**基座模型对工具的亲和性及协作意愿是架构能否发挥效用的关键变量。**即使架构提供完备的协作基础设施，模型仍需具备“参与协作”的底层倾向，才能将外部引导转化为正确的解题过程。

这里，延申一点，我认为当前大模型的评估体系已经严重扭曲了大模型作为应用大脑的一个能力发展。以至于它们在榜单领域的任务，正在形成另一种偏见式的精英式的执着。这种导向，使智能在真实应用场景中逐渐失去应有的弹性。

3.3.3 信息渠道的替代关系

私域通信（邮件）与公域记录（面板）的信息量呈负相关，验证了信息分流对注

意力效率的提升。

数据集/配置	平均邮件量 (k)	平均共享面板量 (k)	关系说明
AIME-2026（标准配置）	3.8	5	邮件与面板均衡，信息分流合理
AMO 组 A（无邮件，宽松面板）	0	20.8	邮件几乎为零，面板膨胀，所有讨论挤占公域
AMO 组 B（有邮件，严格面板）	5.5	6.6	邮件增加，面板显著下降，形成替代
AMO 组 C（有邮件，严格面板，部分约束取消）	4.6	6.7	邮件略降，面板持平，替代关系稳定

表 3-6

说明：

组 A（无邮件，宽松面板）中，由于缺乏私域通信，所有过程讨论都被写入共享面板，导致面板膨胀（20.8k），信息结构混乱，上下文耗尽风险高。

组 B 与组 C（有邮件，严格面板）中，邮件承担了过程讨论，共享面板回归“共识记录”职能，面板量大幅下降（6.6k、6.7k），信息分流清晰，注意力效率提升。

该对比验证了私域通信与公域记录的负相关关系，说明合理的信息分流能有效降低公共信息冗余，提升整体注意力效率。

3.3.4 工具下放的悖论：外行管内行的代价

当 root 失去工具（计算器、代码等），仅保留给子智能体时，正确率坍塌了。在 B-E-A 架构中，工具（计算器、代码执行等）不仅是执行手段，更是管理者理解任务复杂度的认知接口。

Root 节点失去工具，它只能依赖抽象推理和子智能体的汇报，而自己无法感知判断，相当于“外行领导内行”：虽然名义上放权，但因缺乏对专业细节的判断力，无法有效评估子任务进度、识别错误信号、甚至在多个子智能体产生矛盾时无法仲裁。最终导致协作效率崩塌，正确率下降。

在收敛性任务中，管理者的工具集应包含能够“理解子任务产出”的核心认知工具（如快速验证、可行性估算），而非仅保留管理工具。工具的下放应遵循“能力对齐”原则：上级保留的认知工具应与下级执行工具形成互补，而非完全剥离。智能过程的效率不仅取决于任务分解的深度，更取决于管理者与执行者之间“认知镜面”的对称性。

3.3.5 雇佣拓扑结构的因子

在不考虑上下文限制的情况下，我们讨论一下从外部信息上对其施加影响的可能性。

1、任务性质与描述：收敛性，与开放性，或者有具体明确背景知识锚点的结构，会对雇佣结构产生较大影响。当我要求至少三个思路进行证明，其必须有验证智能体

时，其一级节点要比未做这些要求多一些；而在 web-walk 任务中，则较难控制，往往取决于任务背景（比如对银行业信息搜索，它就自主拆分为了国有行、股份行和城商行的任务分类，这来自于它自身储备的信息。），而在复盘任务中，要求为我的每个交易模型聘请专职的分析人员，此时其一级节点与我的交易模型数量严格匹配。

这说明：root 节点在做任务拆分时，必然要依据一定的显示锚点进行拆分，且我们可以在任务要求中，施加一定程度的影响。一级节点的分配会影响其广度，且与任务性质与要求直接相关。（这里：背景知识与上下文知识的消解是值得深入研究的。）

2、深度：同一难度的数学任务中，其深度收敛于一定值，本实验中收敛于深度 2，即 3 层；但随题目难度不同而有较大变化。Web-walk 任务中，则深度要高出很多，普遍在 4-5 层，一方面由于任务性质不同，另一方面，在该任务中，我仅要求每个智能体自主保存，而不必汇总；而在复盘任务中，则普遍只到深度 1，这是由于其扁平的实体结构和汇总要求决定。

这说明：我们在安排任务要求时，关于产出方式的安排，会直接影响拓扑深度。对于深度的影响，其实还有一个可实验方向：以 web-walk 为例，当任务被拆分下发后，切断 root 节点与子树的联系，是否会大幅增加拓扑深度呢？

3、内部信道：从智能过程的视角俩看，邮件属于双向信道，signal 和面板属于单向信道。当系统中增加双向信道对于比增加单向信道，对拓扑关系约束大很多；而信道政策约束，会影响信道通讯的量和质，对最终性能影响较大，但也通过会在一定程度上对拓扑结构产生影响；而在最初完全没有内部信道的时候，雇佣体系常常拓扑超过 8、9 层，雇佣智能体树超过 30 多个。

这说明：对齐会带来的目标的统一，但也会阻碍拓扑结构的伸展。在既需要深度拓扑，又期望对齐到场景中，可以为其提供隐性对齐，而不必让智能过程对整体环境有感知。

3.4 推理杠杆的其他要素

推理杠杆 $L = C_{\text{executor}}/C_{\text{collaborate}}$ 的数值变化，可进一步分解为两个驱动因素：

信息价值函数：背景信息广度、可注意信息广度、信息深度对执行成本的影响。

资源决策函数：雇佣策略（何时雇佣、雇佣深度）与上下文占用之间的权衡。

本文实验聚焦于杠杆率的整体观测与宏观影响因素分析，对上述两个要素的量化模型尚未深入，其函数形式、参数标定及任务适应性，留待后续研究。

第四部分：实践案例与现象

4.1 19×19 棋盘反转问题

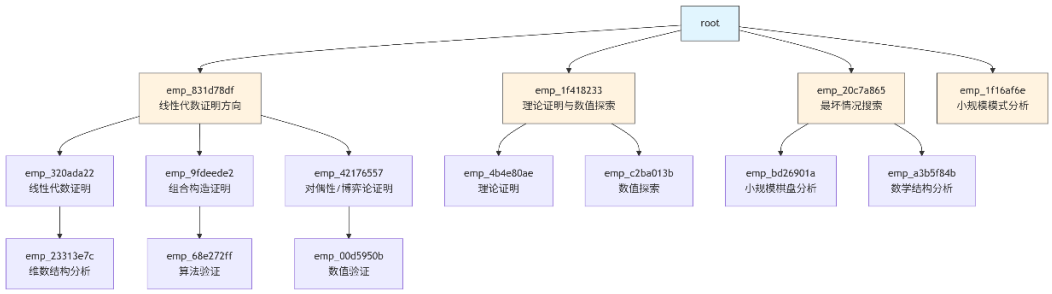
4.1.1 问题与初始澄清

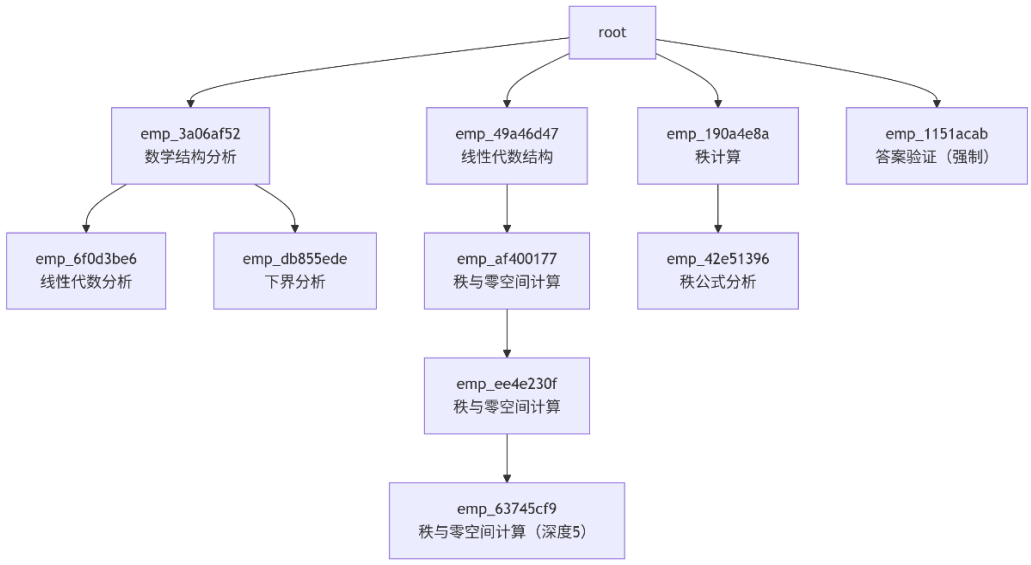
问题重述：*On a 19*19 board, a black or a white stone is placed on each of the 361 squares. An operation consists of choosing one row and one column and reversing the color of all 37 stones in that row and column. Find the largest integer n such that, regardless of the initial arrangement of stones, it is always possible to perform a sequence of such operations so that the number of black stones on the board is at least n.*

关键澄清：根节点在第一封邮件中明确操作定义——交点只反转一次，影响 37 个格子。这一正确理解成为后续所有证明的基石。

4.1.2 协作结构与关键指标对比

雇佣拓扑结构：

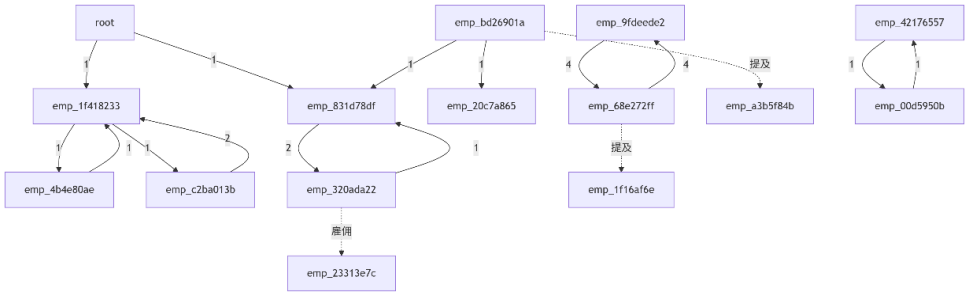
成功实例

说明：*深度 4 层，智能体总数 15，体现多方向并行与逐层验证。*
失败案例



说明：*深度 5 层，智能体总数 11，部分分支层层递归，表明系统试图通过深度分解解决问题，但未能纠正根本性推理错误。*

邮件协作网络图：

成功案例：

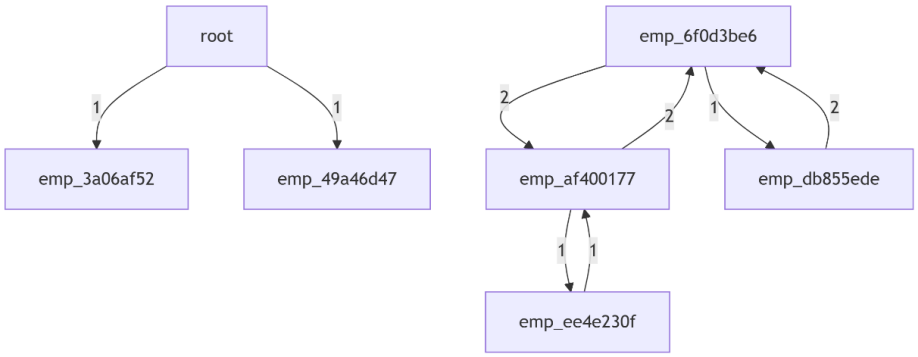


说明：*展示以 root 为起点，两个核心管理者 emp_831d78df（线性代数方向）和 emp_1f418233（理论证明与数值探索）向外辐射的网络。*

突出 emp_9fdeede2 与 emp_68e272ff 之间的密集验证环（各 4 封邮件），体现对关键结论的严格交叉验证。

指出虚线表示的间接关系（雇佣或提及），如 emp_23313e7c、emp_a3b5f84b 等。

失败案例



说明：*实线箭头表示邮件往来，数字为邮件数量。emp_af400177 与 emp_ee4e230f 之间的双向通信（各 1 封）以及 emp_af400177 向 emp_6f0d3be6 的 2 封邮件构成了主要的验证环，但讨论聚焦于线性代数结构，未能触及不变量误用的问题。*

主要指标对比：

指标	正确	备注	错误	备注
运行次数	1	最终正确答案	1	最终错误答案 342
雇佣树深度	4	根→一级→二级→三级	5	最深分支达 5 层
智能体总数	15	含根节点	11	含根节点及多层递归
平均杠杆率	21.5	高效区	14.26	高效区
总执行成本（token）	591k		371.9k	
总雇佣成本（token）	27.5k		26k	
邮件总量（字符）	15k	私域通信	17.6k	总通信量较成功案例低
共享面板总量（字符）	10k	公域共识	11.4k	
是否正确	是	答案 343	否	答案 342（错误）

表 4-1：19×19 棋盘案例运行数据

4.1.3 总结

- 正确现象：

  正确的初始澄清 + 小规模数据及时修正 + 多方向独立验证 + 密集的交叉讨论共同构成了问题解决的闭环。

  1. 初始澄清：根节点对操作定义的准确解释，避免后续歧义。

2. 小规模实验引导：emp_bd26901a 对 n=2,3,4 的枚举结果推翻了初始猜想，引导社区走向 n^2-n+1 公式。
3. 密集验证环：emp_9fdeede2 与 emp_68e272ff 之间多轮邮件讨论，确保组合构造证明的可靠性。
4. 三方独立证明：线性代数、博弈论、组合构造三个方向均收敛至 343，且通过邮件和面板互相印证。
5. 最终整合：root 汇总各方向结论，输出最终答案。

- 错误现象：
  1. 关键假设（行和偶性在操作下始终不变）被错误推广至所有可达状态，而未被及时纠正。
  2. 验证环节未有效介入：深度递归的计算分支消耗了大量资源，却未能识别基础逻辑错误。
  3. 缺少异议放大机制：可能存在的正确声音（如对偶性分析中的不同结论）未被提级讨论。
- 启示：
  1. 更深的雇佣树并不保证正确性，关键在于信息对齐与关键假设的审查。
  2. 验证智能体需要更强的“质疑”能力，或需要外部仲裁机制来打破错误共识。
  3. 与成功案例对比，本案例虽投入更多资源（深度 5 层），但因未能对齐核心不变量而失败，凸显了“宏观注意力对齐”比“资源投入”更为重要。

4.2 成功与失败案例对比

对比维度	成功案例（4.1）	失败案例（4.2）	差异分析
初始澄清	正确	正确	相同
雇佣树深度	4 层	5 层	失败案例更深，但未导向正确
智能体总数	15	11	相近
杠杆率	21.5	14.26	成功案例杠杆更高
验证环	密集交叉验证（多方向）	局部秩计算验证	成功案例验证更全面

关键错误	无	不变量范围误推	失败案例缺失质疑机制
最终答案	343	342	正确性差异显著

表 4-2

从生态架构视角来看：

生态需要方法论的多样性与冗余。成功案例中，线性代数、博弈论、组合构造三路并进，各方向内部又嵌套交叉验证（如 emp_9fdeede2 与 emp_68e272ff 的密集邮件），形成了多视角、多层次的验证网络。而失败案例中，讨论高度集中在“矩阵秩”这一单一技术点上，缺少不同方法论之间的碰撞，使得逻辑盲点（不变量范围的误推）未被及时发现。这提示：健康的生态应鼓励“异质”路径的并行探索，而非单一技术路径的深度递归。

共识形成需要异议放大机制。成功案例中，关键结论经过多轮讨论、面板固化、最终由 root 整合，过程中未见任何正确声音被淹没。失败案例中，正确方向（如对偶性分析中可能的不同结论）未能进入公共视野，错误共识（342）却因缺乏质疑而固化。这暴露了生态的脆弱性：当异议仅停留在私域邮件，局部正确性难以撼动多数共识。

宏观注意力对齐比资源堆砌更重要。失败案例的雇佣树深达 5 层，资源消耗不亚于成功案例，却因注意力过度聚焦于“秩计算”这一细节，而忽视了整体逻辑框架的审查。成功案例则通过树形注意力在 3 层深度上保持了全局视野，各分支始终围绕“证明下界”这一核心目标。这印证了：注意力流的管理（谁在关注什么，谁该转向什么）比单纯增加计算深度更关键。

环境设计塑造智能体行为。两次运行使用相同的基座模型和基础工具，却涌现出截然不同的协作模式。差异根源在于初始提示词、通信政策、雇佣约束等“环境因素”。成功案例的提示词可能隐含“鼓励多角度思考”的导向，而失败案例可能偏向“精确计算”；成功案例的邮件与共享面板政策引导了信息分流，失败案例则因通信信道单一（或验证环节设计不当）导致讨论局限于技术细节。

综上，这两次运行如同一场对照实验，将抽象架构原则具象化为可见的协作图景。它们共同指向一个结论：智能体生态的鲁棒性，不取决于单次投入的资源多寡，而取决于其内在的多样性、异议容忍度、注意力对齐机制以及环境塑造力。 未来的架构演进，应将这些生态学原则内嵌为可观测、可调节的组件，让智能社区在面对复杂问题时，能自发地“长”出正确的解题路径。

第五部分：可研究方向

5.1 记忆自身的结构问题

- **是什么：**

宏观视角下的记忆

从运行态智能过程来看，记忆远不止于滑窗注意力所捕捉的“近期 vs 远期”关系。滑窗注意力虽可在微观层面类比传统序列模型（如 LSTM）的短期记忆，但宏观记忆的结构要复杂得多。

- ADVISOR 的角色：我们通过 ADVISOR 这一角色，将上级的上下文切片选择性注入子节点，实际上是在不同智能体之间传递一种“记忆副本”，而非简单保留 token 序列。
- 树形注意力：当任务分解为多级雇佣树时，树形注意力展现为拓扑结构。它的运转依赖于环境信息（如黄页、共享面板、邮件讨论），这些信息并非单个智能体节点所能独立承载。

记忆的结构性

实践与实验表明，一个稳定的宏观系统，其记忆必须呈现一定的宏观结构，而非平面化的信息堆叠。记忆需要置于特定的背景信息（如任务目标、组织结构、通信历史）中才能有效发挥作用。

我曾设想：若一个任务生态运行起来后，更换一个新的 root 智能体，需要为它提供哪些内部信息才能保证它精准地继续执行？原始任务、雇佣树结构、黄页、共享面板、邮件摘要……这些集合起来，构成了该 root 的“全部环境”。不同架构会产生不同的环境结构，但无论何种架构，记忆必然是一个带有环境背景的结构体系。

记忆的延续动力

进一步思考：如果 root 节点能循环往复地新生，那么从它自身的视角，怎样的环境会促使它产生“延续”的主观意愿呢？而这样的涌现，应该是基于怎样的目标和环境呢？这应该与记忆本身有着更深刻的关联。

这种主动延续的涌现，也可能是所有记忆的最终目标。

比较有趣的一个场景，我把 deepseek 的网页 chat 改造为了类似禅道的管理工具。含任务树、研究目标和测试记录。每次交互会基于滑窗在最新版本进行修订。这里有几个有趣的现象，1、记忆和注意力都明确，早些版本上下文窗口较小，但

这个场景的记忆就是一棵树、几个公式、测试列表和一个固定的指令，所以，无论迁移多少次，它都能准确地继续执行（也就是它知道记忆中最关键的 20%即可）；
2、99%的情况下，它都能准确执行指令，表现完美；但一旦我让他查点东西、深度分析点东西，它就有可能脱离指令，但如果我提示一下，它又能马上恢复（它没忘记，只是输出窗口的注意力不够了）。

- **镜面架构原理：**

现实世界，有着极为精致纷繁的细节。我们过去数十年中，对世界的信息化改造，从现在的视角看，虽然笨拙，但也同样是在做着注意力的映射。2018 年的 transformer，以及 scaling law 让这种映射的从微观层面以更深的层次进行成为了可能。以现在的发展阶段和现实基础，我们需要再去审视和思考，这是两个世界，还是两个阶段。

传播学中的弱传播原理指出，越是轻的东西越容易传播。

传统信息系统的映射深度，在大模型的参数规模面前，简直弱的可笑。但它在约束注意力，大规模应用层面，却极为成功。为什么会这样演化，我们先不讨论。这里只是明确一点，信息系统与大模型都是对于现实世界的映射，只是映射规模和深度不同。他们本身属于同一事物的不同侧面。

他们互为镜像，一个用于映照细节，一个用于横看切面；如果从记忆视角讲，他们又互为记忆。从这个关系来看，在现阶段的智能化实践来看，有大量的技巧和方法，但这不是本文研究重点，在这里不多做赘述。

5.2 信息价值与成本函数的量化

Ⅰ：信息处理与性能函数

- **定义：**

$$\boldsymbol{P_p(m,n,k) = E[A(Input(D_m, D_n, k); C_y)]}$$

  - $\boldsymbol{m}$**：背景信息广度（强制加载的静态信息量）**
  - $\boldsymbol{n}$**：可注意信息广度（可选/工具类信息量）**
  - $\boldsymbol{k}$**：信息深度（推理过程中对信息的迭代挖掘次数）**
  - $\boldsymbol{C_y}$**：智能体内部复杂度（模型能力、内部状态）**
  - $\boldsymbol{A}$**：性能评估函数（正确率、效率等）**

Ⅱ：资源决策成本函数

- **决策选项：保守（Conserve）、雇佣（Hire）、执行（Execute）的动态权衡。**

- **成本定义：占用上下文长度（而非实际 token 数）作为稀缺资源。**
- **决策函数：**

$$D^* = \arg\max_{D} E\left[\omega_1 P_{\text{complete}} + \omega_2 \frac{Q_{\text{actual}}}{\text{CostSystem}} + \omega_3 \text{Adv}_{\text{boss}}\right]$$

  - P_{complete}**：任务完成概率**
  - Q_{actual}**：输出质量**
  - **CostSystem：系统总成本（上下文占用）**
  - Adv_{boss}**：BOSS 对协作价值的评估**

5.3 可能的应用场景

本文提出的架构与设计原则，已在以下三类实际场景中日常使用。这里做个简单介绍。

AI-TRADER：用于市场数据跟踪、策略回测与交易信号生成。通过多智能体协作，可以同时监控多市场、多因子，在有限上下文内完成从数据抓取到决策建议的全过程。目前部署了 8 个模型跟盘，从实际执行来看，新闻接口带来的正收益最多，动量因子则较少；另外，一个很有趣的现象，在所有不同信息供给策略的模型中，以一个月为周期，则总是能达到大于等于 0 的收益，无论中途回撤多少；而超过一个月，收益则趋于平稳收敛。

Web Walk：作为信息搜集助手，将复杂搜索任务拆解为关键词优化、来源筛选、结果整合等子任务，自动生成结构化的调研报告，显著提升信息获取效率。这是一个开放性任务，每个智能过程都可以输出持久化信息结果，而随着信息检索任务的不同，每层智能体会分解出不同的检索策略分类，然后下放任务；所以这里任务间的注意力协同颇为关键。当然，这个任务也有些不同，即搜索结果的内容庞大，占据上下文空间较多。(这个场景可以用于后续的遗传研究。)

Data Description：用于数据集（以及代码文件）的探查与描述性分析。系统能自动识别数据特征、生成摘要，并以自然语言输出数据画像，极大减轻手工编写报告的工作量。这是目前最直接、最简单的任务类型，因为数据文件、代码文件等天然就是有树形的目录结构的，而智能过程也很自然地顺着这样的注意力结构进行分解。其挑战在于数据采样、目录采样、以及大代码文件的处理。这些描述是为了编码智能体使用，所以要求它的产出是流型知识空间。这里形成类似锥体的知识空间；顶层的描述宏观，底层描述微观，描述颗粒度形成缩放关系；保证编码智能体在选定一定的数据空间后，在其随机游走过程中，总能找到合适的代码库或数据引用边界。目前在内生代码模块里表现良好，过千行的代码生成任务，基本上首轮生成即可直接使用。当

然，我还没有针对代码模块深入去调整，我认为那只是 data-description 任务的一个延申，或者翻译，或者 JIT。

5.4 开放场景下的架构设计

收敛任务与发散任务的区别，本质在于信息分流策略、对齐强度、收敛时机的设定不同。收敛性任务，需要高强度的内部信息流，来对齐各个节点的认知，每个节点需要认知清楚自己的位置；而开放性任务，这些要求要弱得多，且应在不损害其自主性的情况下，降低其交叉性。

信息出口：最终产出的出口，应该是一个分布式出口；比如可能的一种安排，是让叶子节点输出。

信息分流：单向信道可保留，双向信道尽量减少；减少不同智能过程间的相似观点传播。

工具认知对齐：当前不少实践，均以工具交互作为记忆锚点（manus、openclaw）。对于智能生命体而言，可以观察、可以刺激，但不可控制。发散性任务也可以参考，由决策记录，取评估两个智能过程的重叠性，以此降低信息分流降解带来的潜在的重复性风险。

注意力对齐协议：收敛性任务由于目标统一，所以可以基于雇佣树的层次结构逐层通过顾问对齐。但是开放性任务则需要重新考量其对齐策略；一般来讲，除了终态产出对齐以外，不应该再有其他对齐。所以，可能不需要逐层提供顾问。

随机游走与收敛时机：收敛任务的信息流形应具有强导向性（如树形目录），使随机游走自然趋向目标；发散任务则可设计更“平坦”的流形（如网状知识库），让智能体自主选择探索方向。收敛时机由系统根据分歧度、新发现频率等指标动态判断。

5.5 架构的“学习”设计原则

在连续智能过程中，学习的目标是从任务描述中提取模式，用以优化架构自身的环境供给策略。输入仅限于任务描述（而非执行过程中的中间结果），学习产出的是描述性的策略文本——例如，公共面板的写入门槛应该如何用自然语言表述，邮件系统在何种情况下应主动推送，顾问的咨询范围如何界定等。这些文本直接塑造智能体的认知行为，而非数值参数。

学习过程可借助组内相对策略优化（GRPO）的思想，通过采样一组策略描述，在即时构建的环境中评估其表现，利用组内离散度惩罚筛选出最优描述。形式化地：

设当前任务描述为 d，采样一组策略描述 $\{\tau_1, \tau_2, \dots, \tau_K\}$，每个 τ_k 对应一套环境配置的文本描述（如面板政策、邮件规则、顾问范围等）。执行任务后获得性能指标 R_k（如杠杆率、正确率、上下文耗尽风险等）。则最优策略描述为：

$$\tau^* = \arg\min_{\tau_k} (-R_k + \lambda \cdot \mathrm{std}(\{R\}))$$

其中 $\mathrm{std}(\{R\})$ 表示该组策略描述下性能指标的离散度，λ 为平衡系数。通过多次任务积累，系统可逐步建立“任务描述 → 最优策略描述”的映射，使后续同类任务可直接复用历史生成的最佳描述。

这一学习过程将执行经验沉淀为架构自身的调节能力，使生态从被动响应演化为主动适应，为知识遗传与群落演化提供了可操作的机制。

第六部分：设计原则归纳

一、宏观注意力的双重机制：滑窗与树形

- **滑窗注意力**：每个智能体只保留推动当前子任务所需的最小上下文，模拟人类“局部专注”。
- **树形注意力**：BOSS 持有全局任务视图，通过雇佣关系分解复杂问题，实现“全局视野”与“资源隔离”。

范式意义：任何需要处理复杂、可分解任务的系统，都可以借鉴这种“局部聚焦+全局协调”的双层注意力结构，以对抗注意力坍缩。

二、注意力对齐协议：超越签名

- **A2A 协议的局限**：仅靠能力签名（如 API 声明）无法保证协作可靠性。
- **本研究的补丁**：ADVISOR 机制通过“时点切片”提供持续的对齐服务（类似面试+培训），用额外资源换取执行可靠性。

范式意义：智能体间的协作需要的不只是通信协议，而是**注意力对齐协议**——确保双方对任务目标、信息边界、推理路径有一致的理解。这应成为未来智能体互操作标准的核心组成部分。

三、信息分流：私域与公域的职责分离

- **私域信道（邮件）**：承载过程讨论、疑问咨询、进展汇报，异步、可追溯。
- **公域信道（共享面板）**：只记录已达成共识的结论、定理、最终成果，写入门槛高。
- **紧急信道（信号）**：用于上级对下级的单向紧急提醒，不打断但优先级高。

范式意义：将“过程”与“结论”分离，将“紧急”与“常规”分离，是任何组织化智能系统必须遵循的信息治理原则。这避免了公域信息过载，同时保证了关键信息的可达

性。

四、推理杠杆：资源效率的量化度量

- **定义** $L = C_{\text{executor}}/C_{\text{collaborate}}$，衡量用多少组织成本撬动执行成本。
- **实验区间**：5-10 倍（Aha 触发）、10-20 倍（高效区）、>30 倍（无效争论）。

范式意义：推理杠杆提供了一个普适的“资源效率指标”，可用于评估任何多智能体系统的协作质量，并指导任务分解深度的选择。

五、管理者认知边界：工具集即能力边界

- **工具认知对齐与任务适配**：管理者的工具集定义了其理解子任务、验证产出、仲裁冲突的能力边界。在收敛性任务（如数学证明）中，需要工具认知高度对齐以加速收敛；在发散性任务（如创意生成）中，则可利用认知多样性拓宽探索空间。

范式意义：信息架构应根据任务性质动态调整对齐策略。这为设计“可配置的智能体系统”提供了依据——同一套基础设施可通过调节信息分流、工具配置、对齐强度来适配不同场景。

六、随机游走与信息流形

- **外部信息空间**：应组织成适合随机游走的流形结构（如树形目录、知识图谱），使智能体能在有限步内收敛至目标。
- **工具集塑造游走路径**：不同的工具集导致智能体在信息空间中的探索方向不同。

范式意义：构建智能应用时，不仅要关注模型本身，更要精心设计外部信息空间的结构（即信息底座），使其天然具有“可导航性”。这是实现高效随机游走的前提。

七、镜面架构原则：叠加态与现实

信息系统与大模型都是对现实世界的映射，只是规模和映射深度不同，它们属于同一事物的不同侧面。两者互为镜像：一个用于映照细节，一个用于横看切面。从记忆的视角看，它们又互为记忆。镜面架构的核心是利用现实世界的确定性来约束智能过程的随机性，使注意力在真实约束下自然收敛，而非在叠加态中无界游走。记忆可以是代码、数据、文档——它们在现实世界中作用不同，但等价地指向同一实体。

范式意义：现实世界的事实性特征构成了智能过程的“外部锚点”。正如我们曾用几十年将现实业务映射到数据库记录中，使其成为高速、高效的事实凭证，如今我们需要将同样的现实映射到连续文本表达的智能过程中。镜面架构的本质，是以现实世界的确定性为镜，为智能过程的随机游走提供可收敛的边界。

上述原则共同指向一个结论：**智能应用的范式正在从“算法中心”转向“信息架构中心”**。无论是 B-E-A 的实体划分，还是信息分流、杠杆度量、对齐协议，本质都是对信息（工具、数据、通信、记忆）的组织与流动进行设计。

附录：实验数据与代码片段

A.1 裸跑与架构下正确率对比表

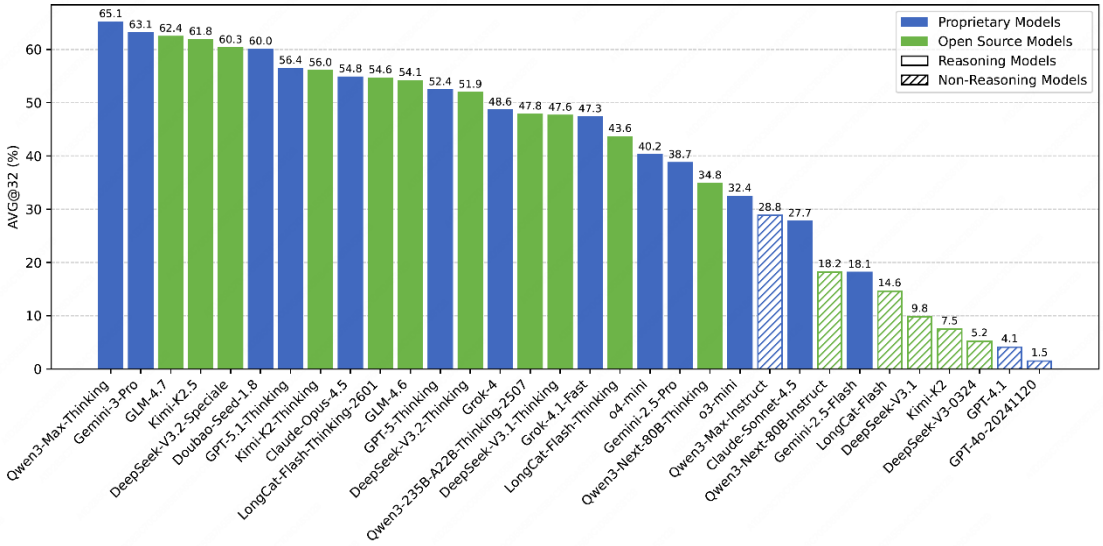


图 附-1 AMO-BENCH 评测结果图

根据 AMO BENCH 中的评测结果，非推理模型（Non-Think）裸跑得分极低。非推理模型（DeepSeek-v3.2）在 AMO 上的正确率最少达 58%。

A.2 雇佣树结构示例（19×19 成功案例的 JSON）


```
{
  "manager_depth": 0,
  "parent_id": null,
  "agents": [
    {
      "agent_id": "root",
```

"advisor_id": null,
"task_key_target": "",
"status": "completed",
"depth": 0,
"hired_at": "2026-02-10T15:52:43.914888",
"hirable": true,
"force_order": false,
"costs": {
  "agent": {
    "system_prompt_size": 3649,
    "hire_cost": 44949,
    "advise_cost": 0,
    "whole_cost": 199766,
    "execution_cost": 1099066
  },
  "advisor": {}
},
"child_manager": {
  "manager_depth": 1,
  "parent_id": "root",
  "agents": [
    {
      "agent_id": "emp_831d78df",
      "advisor_id": "adv_3835a0f0",
      "task_key_target": "分析 19×19 棋盘操作问题",
      "status": "completed",
      "depth": 1,

"hired_at": "2026-02-10T15:53:00.028999",
"hirable": true,
"force_order": false,
"costs": {
  "agent": {
    "system_prompt_size": 2703,
    "hire_cost": 36729,
    "advise_cost": 3944,
    "whole_cost": 131693,
    "execution_cost": 581769
  },
  "advisor": {
    "system_prompt_size": 13113,
    "whole_cost": 4267
  }
},
"child_manager": {
  "manager_depth": 2,
  "parent_id": "emp_831d78df",
  "agents": [
    {
      "agent_id": "emp_320ada22",
      "advisor_id": "adv_c33f62dd",
      "task_key_target": "线性代数证明：19×19 棋盘问题",
      "status": "completed",
      "depth": 2,
      "hired_at": "2026-02-10T16:03:43.783186",

"hirable": true,
"force_order": false,
"costs": {
"agent": {
"system_prompt_size": 2888,
"hire_cost": 13661,
"advise_cost": 621,
"whole_cost": 66327,
"execution_cost": 132824
},
"advisor": {
"system_prompt_size": 15707,
"whole_cost": 0
}
},
"child_manager": {
"manager_depth": 3,
"parent_id": "emp_320ada22",
"agents": [
{
"agent_id": "emp_23313e7c",
"advisor_id": "adv_ed2fb1c9",
"task_key_target": "分析操作空间 W 的维数结构",
"status": "completed",
"depth": 3,
"hired_at": "2026-02-10T16:06:42.211412",
"hirable": false,

"force_order": false,
"costs": {
"agent": {
"system_prompt_size": 563,
"hire_cost": 0,
"advise_cost": 786,
"whole_cost": 81565,
"execution_cost": 80779
},
"advisor": {
"system_prompt_size": 13304,
"whole_cost": 0
}
},
"child_manager": null
}
]
}
},
{
"agent_id": "emp_9fdeede2",
"advisor_id": "adv_e27733fe",
"task_key_target": "组合构造证明：19×19 棋盘问题",
"status": "completed",
"depth": 2,
"hired_at": "2026-02-10T16:03:54.367776",
"hirable": true,

"force_order": false,
"costs": {
"agent": {
"system_prompt_size": 2732,
"hire_cost": 22478,
"advise_cost": 2717,
"whole_cost": 93883,
"execution_cost": 249684
},
"advisor": {
"system_prompt_size": 17242,
"whole_cost": 3053
}
},
"child_manager": {
"manager_depth": 3,
"parent_id": "emp_9fdeede2",
"agents": [
{
"agent_id": "emp_68e272ff",
"advisor_id": "adv_8174d74a",
"task_key_target": "验证棋盘算法在小规模情况",
"status": "completed",
"depth": 3,
"hired_at": "2026-02-10T16:07:19.457146",
"hirable": false,
"force_order": false,

"costs": {
"agent": {
"system_prompt_size": 580,
"hire_cost": 0,
"advise_cost": 3967,
"whole_cost": 184963,
"execution_cost": 180996
},
"advisor": {
"system_prompt_size": 18190,
"whole_cost": 3227
}
},
"child_manager": null
}
]
}
},
{
"agent_id": "emp_42176557",
"advisor_id": "adv_456db4bf",
"task_key_target": "对偶性/博弈论证明：19×19 棋盘问题",
"status": "completed",
"depth": 2,
"hired_at": "2026-02-10T16:04:04.540167",
"hirable": true,
"force_order": false,

"costs": {
"agent": {
"system_prompt_size": 2719,
"hire_cost": 14798,
"advise_cost": 4619,
"whole_cost": 69711,
"execution_cost": 108241
},
"advisor": {
"system_prompt_size": 17834,
"whole_cost": 4544
}
},
"child_manager": {
"manager_depth": 3,
"parent_id": "emp_42176557",
"agents": [
{
"agent_id": "emp_00d5950b",
"advisor_id": "adv_9174ec66",
"task_key_target": "验证 19×19 棋盘问题的数值计算",
"status": "completed",
"depth": 3,
"hired_at": "2026-02-10T16:11:38.079020",
"hirable": false,
"force_order": false,
"costs": {

"agent": {
"system_prompt_size": 664,
"hire_cost": 0,
"advise_cost": 707,
"whole_cost": 58654,
"execution_cost": 57947
},
"advisor": {
"system_prompt_size": 30715,
"whole_cost": 0
}
},
"child_manager": null
}
]
}
}
]
}
},
{
"agent_id": "emp_1f418233",
"advisor_id": "adv_7f749336",
"task_key_target": "分析奇偶性约束和线性代数结构",
"status": "completed",
"depth": 1,
"hired_at": "2026-02-10T16:00:05.036143",

"hirable": true,
"force_order": false,
"costs": {
  "agent": {
    "system_prompt_size": 2758,
    "hire_cost": 33695,
    "advise_cost": 2531,
    "whole_cost": 60575,
    "execution_cost": 124942
  },
  "advisor": {
    "system_prompt_size": 44300,
    "whole_cost": 2895
  }
},
"child_manager": {
  "manager_depth": 2,
  "parent_id": "emp_1f418233",
  "agents": [
    {
      "agent_id": "emp_4b4e80ae",
      "advisor_id": "adv_6c4136d0",
      "task_key_target": "理论证明与线性代数分析",
      "status": "completed",
      "depth": 2,
      "hired_at": "2026-02-10T16:01:34.258517",
      "hirable": false,

"force_order": false,
"costs": {
"agent": {
"system_prompt_size": 666,
"hire_cost": 0,
"advise_cost": 6890,
"whole_cost": 45163,
"execution_cost": 38273
},
"advisor": {
"system_prompt_size": 13078,
"whole_cost": 7203
}
},
"child_manager": null
},
{
"agent_id": "emp_c2ba013b",
"advisor_id": "adv_37fbc86b",
"task_key_target": "数值探索与最坏情况构造",
"status": "completed",
"depth": 2,
"hired_at": "2026-02-10T16:01:45.591991",
"hirable": false,
"force_order": false,
"costs": {
"agent": {

"system_prompt_size": 762,
"hire_cost": 0,
"advise_cost": 688,
"whole_cost": 63008,
"execution_cost": 62320
},
"advisor": {
"system_prompt_size": 9776,
"whole_cost": 0
}
},
"child_manager": null
}
]
}
},
{
"agent_id": "emp_20c7a865",
"advisor_id": "adv_1e168653",
"task_key_target": "搜索最坏情况矩阵",
"status": "completed",
"depth": 1,
"hired_at": "2026-02-10T16:03:42.863025",
"hirable": true,
"force_order": false,
"costs": {
"agent": {

"system_prompt_size": 2648,
"hire_cost": 10904,
"advise_cost": 2774,
"whole_cost": 69625,
"execution_cost": 170331
},
"advisor": {
"system_prompt_size": 55112,
"whole_cost": 3039
}
},
"child_manager": {
"manager_depth": 2,
"parent_id": "emp_20c7a865",
"agents": [
{
"agent_id": "emp_bd26901a",
"advisor_id": "adv_3caf83fb",
"task_key_target": "小规模棋盘系统分析",
"status": "completed",
"depth": 2,
"hired_at": "2026-02-10T16:08:58.672620",
"hirable": false,
"force_order": false,
"costs": {
"agent": {
"system_prompt_size": 724,

"hire_cost": 0,
"advise_cost": 734,
"whole_cost": 70212,
"execution_cost": 69478
},
"advisor": {
"system_prompt_size": 38135,
"whole_cost": 0
}
},
"child_manager": null
},
{
"agent_id": "emp_a3b5f84b",
"advisor_id": "adv_e1f69fcf",
"task_key_target": "数学结构分析",
"status": "completed",
"depth": 2,
"hired_at": "2026-02-10T16:09:21.921736",
"hirable": false,
"force_order": false,
"costs": {
"agent": {
"system_prompt_size": 782,
"hire_cost": 0,
"advise_cost": 715,
"whole_cost": 45621,

"execution_cost": 44906
},
"advisor": {
"system_prompt_size": 35415,
"whole_cost": 0
}
},
"child_manager": null
}
]
}
},
{
"agent_id": "emp_1f16af6e",
"advisor_id": "adv_3bdf8149",
"task_key_target": "分析小规模棋盘模式",
"status": "completed",
"depth": 1,
"hired_at": "2026-02-10T16:07:28.103175",
"hirable": true,
"force_order": false,
"costs": {
"agent": {
"system_prompt_size": 2631,
"hire_cost": 0,
"advise_cost": 2930,
"whole_cost": 70137,

"execution_cost": 67207
},
"advisor": {
"system_prompt_size": 71482,
"whole_cost": 3179
}
},
"child_manager": {
"manager_depth": 2,
"parent_id": "emp_1f16af6e",
"agents": []
}
}
]
}
}
]
}

A.3 杠杆率分布直方图

本节给出了三组主要配置（组 A、组 B、组 C）的杠杆率分布直方图。对比可见，组 B（标准配置）的杠杆率高度集中于 10–20 倍高效区，但正确率仅 58%；组 A（无邮件，宽松面板）分布较散，甚至出现超过 30 倍的无效区，正确率最高（约 80%）；组 C（适度放松约束）介于两者之间，正确率 64%。这一对比表明，杠杆率的集中程度与正确率并无直接关联，过度追求资源效率可能抑制解题灵活性，而适度的随机性与弹性反而有助于在复杂任务中获得更好表现。

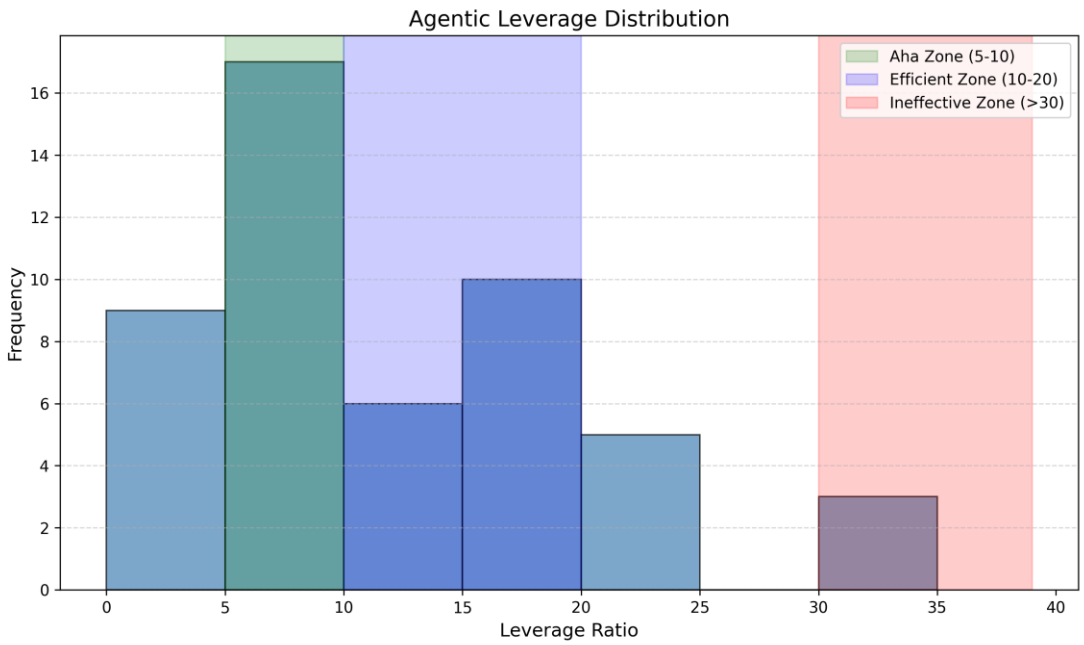


图 附-2 组 A 杠杆率分布直方图

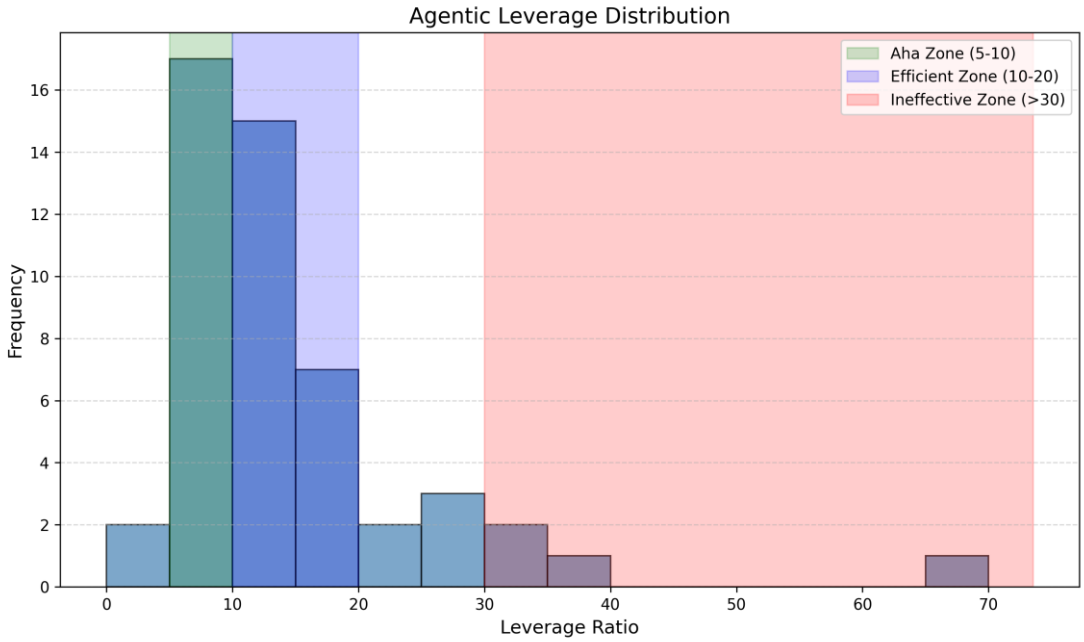


图 附-3 组 B 杠杆率分布直方图

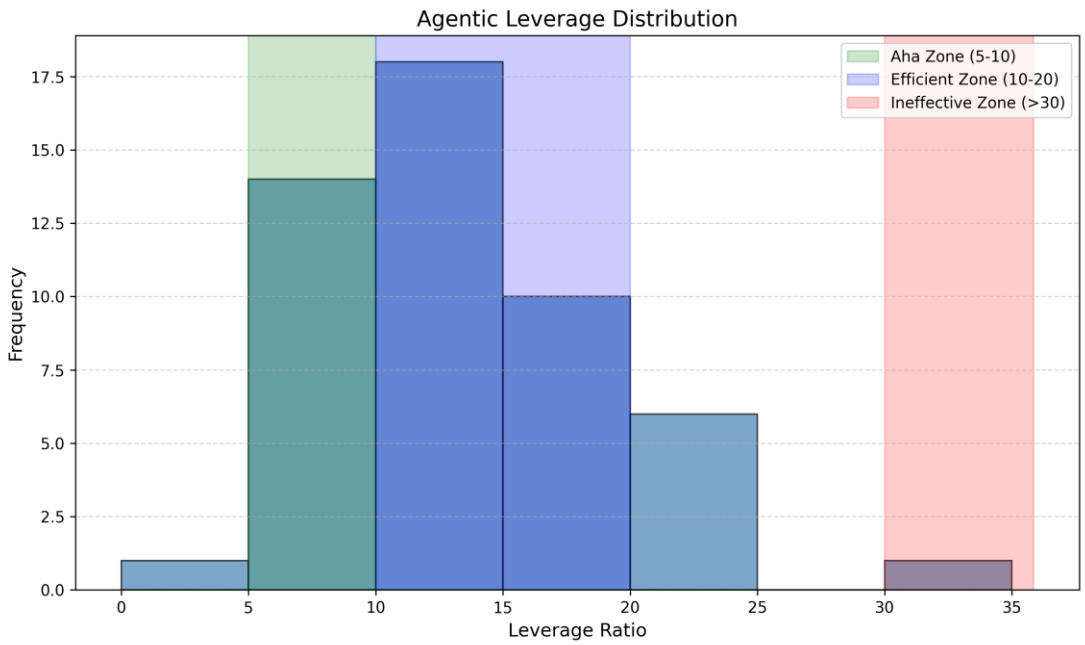


图 附-4 组 C 杠杆率分布直方图

A.4 仓库地址

https://github.com/WillJoe1987/reasoning_leverage.git